

```
-- =====  
-- AMAZON BRAZIL SQL PROJECT
```

```
-- Name : Ranjeet Anada  
-- =====
```

```
-- =====  
-- Q1. Find the total number of orders fulfilled by each seller state.  
-- =====
```

```
SELECT  
  
    s.seller_state,  
  
    COUNT(DISTINCT oi.order_id) AS total_orders  
  
FROM sellers s  
  
JOIN order_items oi  
  
ON s.seller_id = oi.seller_id  
  
GROUP BY s.seller_state  
  
ORDER BY total_orders DESC;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Contents:
seller_state	total_orders			
SP	70188			
MG	7930			
PR	7673			
RJ	4353			
SC	3667			
RS	1989			
DF	824			
BA	569			
GO	463			
PE	406			
MA	392			
ES	318			
MT	137			
CE	91			
RN	51			
MS	49			
PB	36			
RO	14			
PI	12			
SE	9			
PA	8			
AM	3			
AC	1			

Result 1 x

```
-- Q2. For each product category, calculate the cumulative revenue generated as orders
come in over time.
```

```
SELECT

    p.product_category_name,

    DATE(o.order_purchase_timestamp) AS order_date,

    SUM(oi.price) AS daily_revenue,

    SUM(SUM(oi.price)) OVER (

        PARTITION BY p.product_category_name

        ORDER BY DATE(o.order_purchase_timestamp)

    ) AS cumulative_revenue

FROM products p

JOIN order_items oi

ON p.product_id = oi.product_id

JOIN orders o

ON oi.order_id = o.order_id

GROUP BY

    p.product_category_name,

    DATE(o.order_purchase_timestamp);
```

product_category_name	order_date	daily_revenue	cumulative_revenue
MULE	2016-10-04	49.99	49.99
MULE	2016-10-08	15.9	65.89
MULE	2017-01-17	169	234.89
MULE	2017-01-20	349.99	584.88
MULE	2017-01-25	378.9	963.78
MULE	2017-01-26	3.9	378.9168
MULE	2017-01-27	602.79	1570.4699999999998
MULE	2017-01-28	84.8	1655.2699999999998
MULE	2017-01-29	90	1745.2699999999998
MULE	2017-01-31	23.26	1768.5299999999997
MULE	2017-02-01	273	2041.5299999999997
MULE	2017-02-02	77.99	2119.5199999999995
MULE	2017-02-03	185.7	2305.2199999999993
MULE	2017-02-04	4518.98	6824.199999999999
MULE	2017-02-05	283.24	7107.439999999999
MULE	2017-02-06	764.79	7872.229999999999
MULE	2017-02-07	412.49	8284.72
MULE	2017-02-08	304.6	8589.32
MULE	2017-02-09	172.9800000...	8762.3
MULE	2017-02-10	49.8	8812.099999999999
MULE	2017-02-11	125.85	8937.949999999999

-- =====

-- Q3. Which payment method do customers use the most, and what is the average order value for each payment type?

-- =====

```
SELECT
    payment_type,
    COUNT(*) AS total_transactions,
    ROUND(AVG(payment_value),2) AS average_order_value
FROM order_payments
GROUP BY payment_type
ORDER BY total_transactions DESC;
```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 			
	payment_type	total_transactions	average_order_value
▶	credit_card	76795	163.32
	boleto	19784	145.03
	voucher	5775	65.7
	debit_card	1529	142.57
	not_defined	3	0

Result 3 x

-- =====

-- Q4. Find the customer who has spent the most money across all their orders.

-- =====

```
SELECT
    o.customer_id,
    SUM(op.payment_value) AS total_spent
FROM orders o
JOIN order_payments op
    ON o.order_id = op.order_id
GROUP BY o.customer_id
ORDER BY total_spent DESC
LIMIT 1;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	customer_id	total_spent				
▶	1617b1357756262bfa56ab541c47bc16	13664.08				

Result 4 x

-- =====
-- Q5. Find the average review score for each product category.
-- =====

SELECT

p.product_category_name,

ROUND(AVG(r.review_score), 2) AS average_review_score

FROM products p

JOIN order_items oi

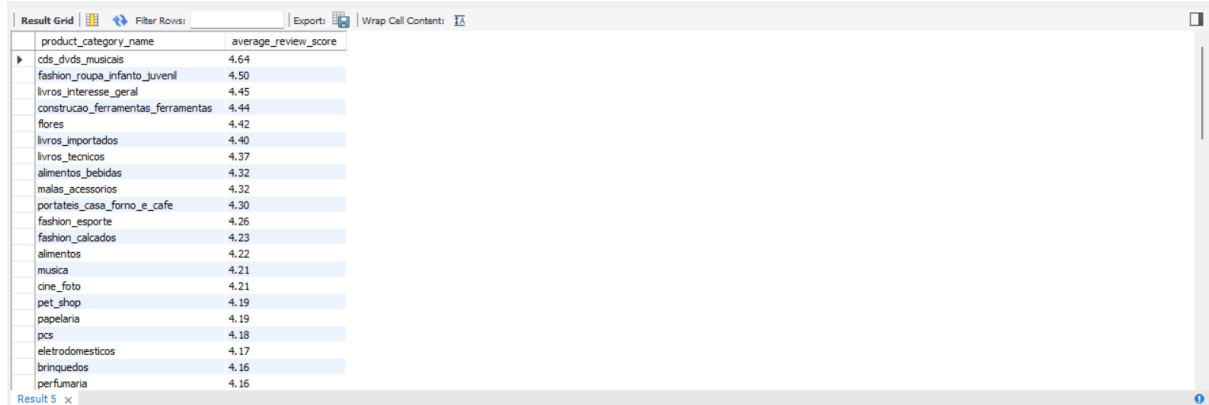
ON p.product_id = oi.product_id

JOIN order_reviews r

ON oi.order_id = r.order_id

GROUP BY p.product_category_name

ORDER BY average_review_score DESC;



The screenshot shows a database query result grid with two columns: 'product_category_name' and 'average_review_score'. The results are ordered by the average review score in descending order. The categories and their scores are as follows:

product_category_name	average_review_score
cds_divds_musicais	4.64
fashion_roupa_infanto_juvenil	4.50
livros_interesse_geral	4.45
construcao_ferramentas_ferramentas	4.44
flores	4.42
livros_importados	4.40
livros_tecnicos	4.37
alimentos_bebidas	4.32
malas_acessorios	4.32
portateis_casa_forno_e_cafe	4.30
fashion_esporte	4.26
fashion_calcados	4.23
alimentos	4.22
musica	4.21
cine_foto	4.21
pet_shop	4.19
papelaria	4.19
pcs	4.18
eletrrodomesticos	4.17
brinquedos	4.16
perfumaria	4.16

```
-- =====
-- Q6. Find the total number of orders placed by each customer,
-- broken down by the state they live in.
-- =====
```

```
SELECT

    c.customer_state,

    c.customer_id,

    COUNT(o.order_id) AS total_orders

FROM customers c

JOIN orders o

    ON c.customer_id = o.customer_id

GROUP BY c.customer_state, c.customer_id

ORDER BY total_orders DESC;
```

customer_state	customer_id	total_orders
SP	c9cb49149d6cde01cc34c008d9f7ab76	1
RJ	9e27532e607078fbf6344b7c1063e65	1
SC	cff7d309b878ba3fc6bc7093c68fb8c7	1
MG	e73a40f7509a5e81a764029d4b7ea836	1
BA	1964a80794004a2f2456740630cc3b9d	1
RJ	568be7bdad827e05c5b0b87614cb8c59	1
MG	f53789fc4a603015c6e5a040fffb6cac	1
SP	b49cc676663ad8545fd56c4c2ec8545d	1
MG	b58289fc45b5ab5b1ce95864818e98e6	1
SC	fa70d6b1e9a73119bbbf5c9bffb84b5e	1
SP	06b1837a8f2cabe23495d0c93519bc26	1
RJ	5db443877524f04312886fad87e08e49	1
SP	4bdf8269e9f0f30a5f8c14e9df179854	1
SP	5b99d2ac45d9e913fa37e13a7e73f1fc	1
SP	d06ae9dcd2fca7751cf48ef8ed08b50c	1
ES	08dc787bd670738dde6d2b5302c922f	1
BA	d1ab1355d889d31e0140106dead20579	1
RJ	56d173fa525c87c279927cc4e62eba2f	1
SP	89ac41e89828b6d6a8297c56ac839e05	1
RJ	ca1a7da184d06f128df4600dd61381e0	1
SP	c4f1520e30ecacaf4625b73784fa4c92	1

-- Q7. Identify sellers who registered on the platform
-- but have never fulfilled a single order.

```
SELECT
    s.seller_id,
    s.seller_state
FROM sellers s
LEFT JOIN order_items oi
    ON s.seller_id = oi.seller_id
WHERE oi.order_id IS NULL;
```

-- Q8. Find the top 5 product categories by total revenue.

```
SELECT
    p.product_category_name,
    SUM(oi.price) AS total_revenue
FROM products p
JOIN order_items oi
    ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Result Grid		Filter Rows:	Exports:	Wrap Cell Content:	Fetch rows:
	product_category_name	total_revenue			
▶	beleza_saude	1258681.3399999454			
	relogios_presentes	1205005.6799999997			
	cama_mesa_banho	1036988.6800000712			
	esporte_lazer	988048.9700000435			
	informatica_acessorios	911954.3200000289			

Result 8 x

-- =====

-- Q9. Calculate the median delivery time for orders.

-- =====

WITH delivery_time AS (

SELECT

DATEDIFF(order_delivered_customer_date, order_purchase_timestamp) AS
delivery_days

FROM orders

WHERE order_delivered_customer_date IS NOT NULL

)

SELECT

AVG(delivery_days) AS median_delivery_days

FROM (

SELECT

delivery_days,

ROW_NUMBER() OVER (ORDER BY delivery_days) AS rn,

COUNT(*) OVER () AS total_rows

FROM delivery_time

) t

WHERE rn IN ((total_rows + 1) / 2, (total_rows + 2) / 2);

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	median_delivery_days			
▶	10.0000			

Result 9 x


```
-- Q11. Find sellers whose total number of orders is above
-- the average number of orders fulfilled by all sellers.
```

```
SELECT
    seller_id,
    COUNT(order_id) AS total_orders
FROM order_items
GROUP BY seller_id
HAVING COUNT(order_id) >
(
    SELECT AVG(order_count)
    FROM (
        SELECT COUNT(order_id) AS order_count
        FROM order_items
        GROUP BY seller_id
    ) avg_orders
);
```

seller_id	total_orders
48436dade18ac8b2bce089ec2a041202	151
dd7ddc04e1b6c2c614352b383efe2d36	143
7040e82f899a04d1b434b795a43b4617	228
a416b6a846a11724393025641d4edd5e	181
ba143b05f0110f0dc71ad71b4466ce92	86
cc419e0650a3c5ba77189a1882b7556a	1775
4a3ca9315b744ce9f8e9374361493884	1987
1f50f920176fa81dab994f9023523100	1931
530ec6109d11eaa87999465c6afee01	104
fcb5ace8bcc92f75707dc0f01a27d269	73
fe2032dab1a61af8794248c196565c9	306
218d46b86c1881d022bce9c68a7d4b15	427
8dbac7e12637ed9cffa18c7875207478	82
1c129092bf23f28a5930387c980c0dfc	220
ea8482cd71df3c1969d7b9473ff1abc	1203
7c67e1448b00f6e969d365cea6b010ab	1364
6560211a19b47992c3666cc4a7e94c0	2033
3504c0db71d7fa48d967e04c94d59d9	53
c864036faab8c1659f65ea4fae61da	58
bfd279966d91cfaef8b25d076585f0da	118
41b39e28db005d9731d9d485a83b4c38	231

```
-- =====  
-- Q12. Find the average review score for each customer state.  
-- =====
```

```
SELECT
```

```
    c.customer_state,
```

```
    ROUND(AVG(r.review_score), 2) AS average_review_score
```

```
FROM customers c
```

```
JOIN orders o
```

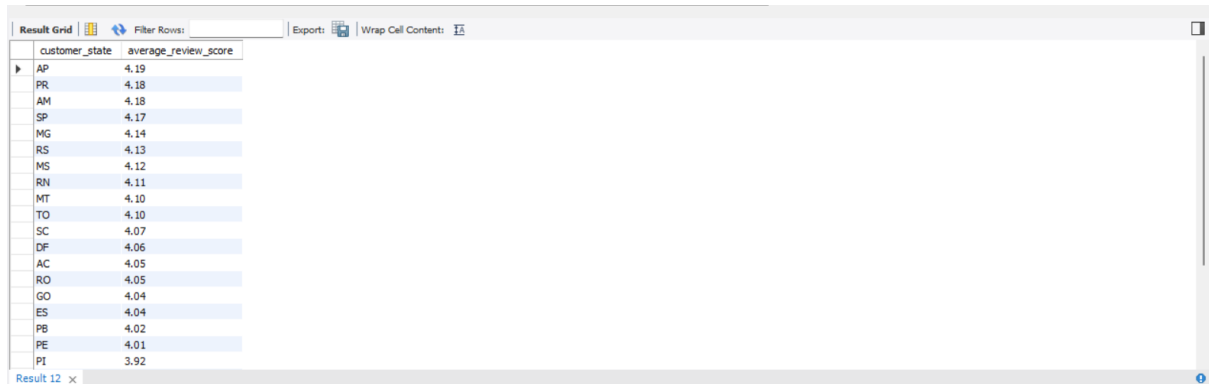
```
    ON c.customer_id = o.customer_id
```

```
JOIN order_reviews r
```

```
    ON o.order_id = r.order_id
```

```
GROUP BY c.customer_state
```

```
ORDER BY average_review_score DESC;
```

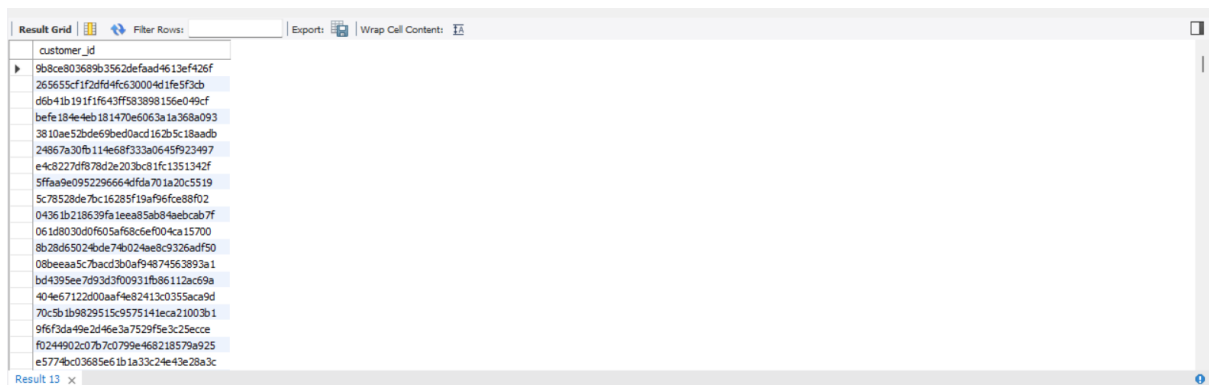


The screenshot shows a database query result grid with two columns: 'customer_state' and 'average_review_score'. The results are ordered by the average review score in descending order. The states and their scores are: AP (4.19), PR (4.18), AM (4.18), SP (4.17), MG (4.14), RS (4.13), MS (4.12), RN (4.11), MT (4.10), TO (4.10), SC (4.07), DF (4.06), AC (4.05), RO (4.05), GO (4.04), ES (4.04), PB (4.02), PE (4.01), and PT (3.92).

customer_state	average_review_score
AP	4.19
PR	4.18
AM	4.18
SP	4.17
MG	4.14
RS	4.13
MS	4.12
RN	4.11
MT	4.10
TO	4.10
SC	4.07
DF	4.06
AC	4.05
RO	4.05
GO	4.04
ES	4.04
PB	4.02
PE	4.01
PT	3.92

```
-- =====  
-- Q13. Find customers who have placed orders but never  
-- submitted a review.  
-- =====
```

```
SELECT DISTINCT  
    c.customer_id  
FROM customers c  
JOIN orders o  
    ON c.customer_id = o.customer_id  
LEFT JOIN order_reviews r  
    ON o.order_id = r.order_id  
WHERE r.review_id IS NULL;
```



The screenshot shows a database query result grid with a single column titled 'customer_id'. The grid contains 20 rows of hexadecimal strings, which are UUIDs. The interface includes a 'Result Grid' header, a 'Filter Rows' button, and an 'Export' button. The status bar at the bottom indicates 'Result 13'.

customer_id
9b8ce803689b3562defaad4613ef426f
265655cf1f2df44fc630004d1fe5f3cb
d6b41b191f1f643ff583898156e049cf
befe184e4eb181470e6063a1a368a093
3810ae52bde69bed0acd162b5c18aad5
24867a30fb114e68f333a0645f923497
e4c8227d878d2e203bc81fc1351342f
5ffa9e0952296664dfda701a20c5519
5c78528de7bc16285f19af96fce88f02
04361b218639fa1eea85ab84aebcab7f
061d8030d0f605af68c6ef004ca15700
8b28d65024bde74b024ae8c9326adff50
08beaa5c7bacd3b0af94874563893a1
bd4395ee7d93d3f00931fb86112ac69a
404e6712d00aaf4e82413c0355aca9d
70c5b1b9829515c9575141eca21003b1
9f6f3da49e2d46e3a7529f5e3c25ecce
f0244902c07b7c0799e468218579a925
e5774bc03685e61b1a33c24e43e28a3c

-- =====

-- Q14. Find the month with the highest number of orders.

-- =====

SELECT

MONTHNAME(order_purchase_timestamp) AS month_name,

COUNT(order_id) AS total_orders

FROM orders

GROUP BY

MONTH(order_purchase_timestamp),

MONTHNAME(order_purchase_timestamp)

ORDER BY total_orders DESC

LIMIT 1;

Result Grid		Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	month_name	total_orders			
▶	August	10843			

Result 14 x